Laboratorio 2 — Semáforos en XV6

Clase 1: vistazo al lab e introducción a xv6

Sistemas Operativos — FaMAF, UNC

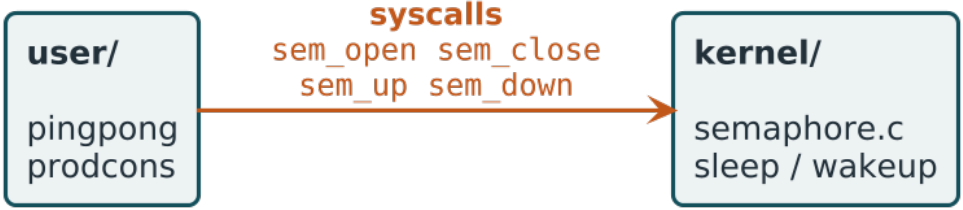
15/09/2026

Agenda

- Lab 2: presentación
- xv6
- Viendo el código

Lab 2

Qué van a construir (de punta a punta)



Las cuatro piezas

1. **4 syscalls** en el kernel: `sem_open`, `sem_close`, `sem_up`, `sem_down`
2. **`pingpong`** — dos procesos alternan mensajes
   (entrega intermedia)
3. **`prodcons`** — productor–consumidor con buffer acotado
   (entrega final)
4. *(Opcional)* Punto estrella: varios productores y consumidores

Fechas y auto-corrección

Entrega	Qué entregan	Cuándo	Comando
Intermedia	Kernel + `pingpong`	**28/09** 23:59	`make grade-parcial` → **11/11**
Final	+ `prodcons` + video	**07/10** 23:59	`make grade-final` → **15/15**

Qué reciben hoy

Material	**Para qué**
Enunciado (PDF)	Especificación completa
`lab2-kickstarter.tgz`	Tests automáticos + `semtest`
Repo Bitbucket del grupo	Donde hacen push

Qué es xv6

xv6: de dónde viene

1975	**Unix V6** (Bell Labs) — Ken Thompson y Dennis Ritchie
1976	**Lions' Commentary**: el código de V6 comentado línea por línea. AT&T restringió su distribución y circuló fotocopiado 20 años.
2006	**MIT** reimplementa V6 como **xv6**: mismo diseño, pero en ANSI C y para multiprocesador (curso 6.828).
2019	Port a **RISC-V** — es el que vamos a usar.

El kernel entero son $\sim$4.400 líneas de C en 25 archivos. Está pensado para **leerlo completo**, no para usarlo.

Un detalle local

En los agradecimientos del `README` de xv6 aparecen

Nicolas Wolovick y Pablo Ventura

Una frase

xv6 es un sistema operativo didáctico
(Unix simplificado) para **RISC-V**.

Corre en QEMU — un emulador.
No necesitan hardware real.

Capas


user/ ls, sh, pingpong, …
programas de usuario
kernel/ syscalls, procesos, VM
una sola imagen, privilegiada
QEMU / RISC-V
emula la máquina

Los dos modos

- **User mode:** procesos aislados; no acceden a memoria del kernel directamente.
- **Kernel mode:** schedulea procesos, atiende interrupciones, implementa syscalls.
- **Comunicación:** sólo vía system calls (`write`, `fork`, `exit`, y las nuevas `sem_*`).

Conceptos xv6 que ya usaron (o verán hoy)

Idea	En la práctica
`fork()`	Crear un proceso hijo (copia del padre)
`wait()`	Padre espera que el hijo termine
`exit()`	Terminar el proceso
Shell `$`	Loop que lee comandos y los ejecuta

`pingpong` y `prodcons` usan `fork`: padre e hijo son procesos distintos que comparten semáforos globales del kernel.

Commit base del lab

Usar xv6 en el commit `f5b93ef`

- No mezclar con ramas de otros laboratorios.

Cómo se desarrolla (ciclo diario)

```
make qemu          # compila y abre xv6 en QEMU
# probar: $ ls, $ echo hola
# salir: Ctrl-a x
make grade-parcial # auto-tests (fallará hasta que implementen)
```

Setup práctico (demo en vivo)

Setup práctico

Pasamos a la terminal

- Clonar xv6 y armar el repo del grupo.
- Compilar y entrar: `make qemu`.
- Probar el shell: `ls`, `echo`, `cat`.
- Aplicar el kickstarter y ver qué agrega.

Qué trae el kickstarter

- `user/semtest.c` — prueba las syscalls (no es la solución).
- Scripts `grade-lab2-*` y targets `make grade-parcial` / `make grade-final`.
- No trae el kernel implementado ni `pingpong`/`prodcons` completos.

Salir de QEMU

Ctrl-a, soltar, **x**.

Problemas frecuentes

Problema	**Solución típica**
`qemu-system-riscv64: command not found`	Instalar qemu
Error de compilación tras el patch	Revisar que el patch se aplicó en el directorio correcto
Pantalla negra	Esperar unos segundos; verificar que `make` terminó sin errores

Mapa rápido del código

Carpetas principales

```
xv6-riscv/
├── kernel/          ← acá implementan semáforos (semaphore.c)
├── user/            ← pingpong.c, prodcons.c, semtest.c
├── Makefile
└── ...
```

Archivos que van a tocar

Zona	Archivos clave
Kernel nuevo	`kernel/semaphore.c`, `semaphore.h`
Registrar syscalls	`syscall.h`, `syscall.c`, `sysproc.c`, `main.c`
User API	`user/user.h`, `user/usys.pl`
Programas	`user/pingpong.c`, `user/prodcons.c`

(sólo nombres por ahora)

Flujo simplificado (para la clase 2)

`pingpong.c` → `user.h` → `usys.S` → `kernel/syscall.c` → `sysproc.c` → `semaphore.c`

- Hoy alcanza con saber que existe ese camino.
- En la clase 2 lo vemos con más detalle.

Qué leer del enunciado esta semana

1. Introducción (contexto del lab).
2. Syscalls — tabla de `sem_open`, `sem_close`, `sem_up`, `sem_down`.
3. Políticas obligatorias (ids 1–64, retorno 0 = error).
4. Programa `pingpong` (sin implementar aún).

Obligatorio antes de la próxima clase

- ☐ Repo del grupo con xv6 en `f5b93ef` + kickstarter aplicado.
- ☐ `make qemu` entra al shell sin errores.
- ☐ Leer enunciado: Intro + Syscalls + `pingpong`.
- ☐ *Opcional:* correr `make grade-parcial` y ver que falla (baseline).

Camino a la intermedia (clases 1–4)

Clase	Fecha	Qué vemos
1	15/09	xv6 + setup
2	17/09	Semáforos + anatomía de syscalls
3	22/09	Kernel `sem_*` + diseño de `pingpong`
4	24/09	Cierre `pingpong` + debugging
Entrega	**28/09 23:59**	`make grade-parcial` → **11/11**

Pregunta para traer pensada

Si dos procesos quieren imprimir al mismo tiempo, ¿qué puede salir mal?

Referencias

- **xv6 book** — cap. 0 (intro), cap. 2 (syscalls, lectura liviana).
- **Enunciado** — secciones *Introducción* y *Cómo probar*.